

Week 2: MLFQ Scheduler Implementation

Core Scheduler Logic and Demotion Policy

Abdullah Khan Sherwani 27880

Muhammad Anas Tabba 28998

3rd December 2025

Contents

1	Overview	2
2	Scheduler Implementation	2
2.1	MLFQ Scheduler Logic (kernel/proc.c)	2
2.2	Priority Boost Function	3
3	Time Slice Tracking and Demotion	3
3.1	Timer Interrupt Handler (kernel/trap.c)	3
3.2	Demotion Thresholds	3
4	Files Modified	4
5	Testing	4
5.1	CPU-Bound Process Behavior	4
5.2	I/O-Bound Process Behavior	4
5.3	Fairness Test	5
6	Summary	6

1 Overview

Building on Week 1's design, Week 2 implements the core MLFQ scheduling logic, time slice tracking, and automatic demotion policy.

Objectives: Implement priority-based scheduling in `scheduler()`, add per-tick time slice tracking, implement automatic demotion, and create CPU-bound/I/O-bound test programs.

2 Scheduler Implementation

2.1 MLFQ Scheduler Logic (kernel/proc.c)

```
1 void scheduler(void) {
2     struct proc *p;
3     struct cpu *c = mycpu();
4     c->proc = 0;
5
6     for(;;){
7         intr_on();
8         intr_off();
9
10        // Priority boost check (starvation prevention)
11        acquire(&tickslock);
12        uint64 current_ticks = ticks;
13        if(current_ticks - last_boost_time >= BOOST_INTERVAL) {
14            last_boost_time = current_ticks;
15            release(&tickslock);
16            boost_all_priorities();
17        } else {
18            release(&tickslock);
19        }
20
21        int found = 0;
22        // Scan queues from highest (0) to lowest (3)
23        for(int priority = 0; priority < NMLFQ && !found; priority++)
24        {
25            for(p = proc; p < &proc[NPROC]; p++) {
26                acquire(&p->lock);
27                if(p->state == RUNNABLE && p->priority == priority) {
28                    p->state = RUNNING;
29                    c->proc = p;
30                    swtch(&c->context, &p->context);
31                    c->proc = 0;
32                    found = 1;
33                    release(&p->lock);
34                    break;
35                }
36                release(&p->lock);
37            }
38        }
39    }
```

```

38     if(!found) asm volatile("wfi");
39 }
40 }

```

2.2 Priority Boost Function

```

1 void boost_all_priorities(void) {
2     struct proc *p;
3     for(p = proc; p < &proc[NPROC]; p++) {
4         acquire(&p->lock);
5         if(p->state == RUNNABLE || p->state == RUNNING || p->state ==
        SLEEPING) {
6             p->priority = 0;
7             p->time_slices = 0;
8         }
9         release(&p->lock);
10    }
11 }

```

3 Time Slice Tracking and Demotion

3.1 Timer Interrupt Handler (kernel/trap.c)

```

1 // In usertrap(), on timer interrupt (which_dev == 2):
2 p->time_slices++;
3
4 extern int mlfq_time_quanta[];
5 #define NMLFQ 4
6
7 if(p->time_slices >= mlfq_time_quanta[p->priority]) {
8     if(p->priority < NMLFQ - 1) {
9         p->priority++; // Demote to lower priority
10    }
11    p->time_slices = 0; // Reset for new queue
12 }
13 yield();

```

3.2 Demotion Thresholds

Queue	Quantum	Demotes To
Q0	2 ticks	Q1
Q1	4 ticks	Q2
Q2	8 ticks	Q3
Q3	16 ticks	Stays Q3

Table 1: Demotion Thresholds

4 Files Modified

- kernel/proc.c – MLFQ scheduler logic and boost function
- kernel/trap.c – Time slice tracking and demotion
- user/cpubound.c, iobound.c, mlfqtest.c – Test programs
- Makefile – Added test programs to UPROGS

5 Testing

5.1 CPU-Bound Process Behavior

CPU-intensive processes consume their time slices and get demoted from $Q0 \rightarrow Q1 \rightarrow Q2 \rightarrow Q3$.

```

13 cpubound
=== Comprehensive CPU-Bound Test ===
This process will run CPU-intensive work for ~38 seconds.
When priority change from Q0 -> Q1 -> Q2 -> Q3
Starting: PID=7, Priority=Q0, TimeSlices=8
Running continuous computation (no sleeps, no syscalls)...
Expected demotions:
0-2 ticks : Q0 (initial)
2-4 ticks : Q1 (after first demotion)
4-14 ticks : Q2 (after second demotion)
14+ ticks : Q3 (after third demotion)

Checkpoint 0: Priority=Q0, TimeSlices=8
Checkpoint 1: Priority=Q0, TimeSlices=1
Checkpoint 2: Priority=Q1, TimeSlices=4
Checkpoint 3: Priority=Q1, TimeSlices=4
Checkpoint 4: Priority=Q1, TimeSlices=1
Checkpoint 5: Priority=Q1, TimeSlices=2
Checkpoint 6: Priority=Q1, TimeSlices=3
Checkpoint 7: Priority=Q1, TimeSlices=4
Checkpoint 8: Priority=Q2, TimeSlices=4
Checkpoint 9: Priority=Q2, TimeSlices=1
Checkpoint 10: Priority=Q2, TimeSlices=1
Checkpoint 11: Priority=Q2, TimeSlices=1
Checkpoint 12: Priority=Q2, TimeSlices=2
Checkpoint 13: Priority=Q2, TimeSlices=3
Checkpoint 14: Priority=Q2, TimeSlices=4
Checkpoint 15: Priority=Q2, TimeSlices=4
Checkpoint 16: Priority=Q2, TimeSlices=5
Checkpoint 17: Priority=Q2, TimeSlices=6
Checkpoint 18: Priority=Q2, TimeSlices=6
Checkpoint 19: Priority=Q2, TimeSlices=7
Checkpoint 20: Priority=Q3, TimeSlices=8
Checkpoint 21: Priority=Q3, TimeSlices=8
Checkpoint 22: Priority=Q3, TimeSlices=1
Checkpoint 23: Priority=Q3, TimeSlices=2
Checkpoint 24: Priority=Q3, TimeSlices=2
Checkpoint 25: Priority=Q3, TimeSlices=3
Checkpoint 26: Priority=Q3, TimeSlices=3
Checkpoint 27: Priority=Q3, TimeSlices=4
Checkpoint 28: Priority=Q3, TimeSlices=4
Checkpoint 29: Priority=Q3, TimeSlices=5
Checkpoint 30: Priority=Q3, TimeSlices=6
(Continuing in background for full test...)
Checkpoint 31: Priority=Q3, TimeSlices=7
Checkpoint 32: Priority=Q3, TimeSlices=7
Checkpoint 33: Priority=Q3, TimeSlices=8
Checkpoint 34: Priority=Q3, TimeSlices=9
Checkpoint 35: Priority=Q3, TimeSlices=9
Checkpoint 36: Priority=Q3, TimeSlices=10
Checkpoint 37: Priority=Q3, TimeSlices=11
Checkpoint 38: Priority=Q0, TimeSlices=8
Checkpoint 39: Priority=Q0, TimeSlices=8
Checkpoint 40: Priority=Q0, TimeSlices=1
Checkpoint 41: Priority=Q1, TimeSlices=8
Checkpoint 42: Priority=Q1, TimeSlices=1
Checkpoint 43: Priority=Q1, TimeSlices=1
Checkpoint 44: Priority=Q1, TimeSlices=2
Checkpoint 45: Priority=Q1, TimeSlices=3
Checkpoint 46: Priority=Q1, TimeSlices=3
Checkpoint 47: Priority=Q2, TimeSlices=8
Checkpoint 48: Priority=Q2, TimeSlices=1
Checkpoint 49: Priority=Q2, TimeSlices=2
Final Result: Priority=Q2, TimeSlices=2
✓ Q000: Reached Q2 (mid-range demotion)

```

Figure 1: CPU-bound process demotion through queue levels

5.2 I/O-Bound Process Behavior

I/O-bound processes yield before exhausting their quantum and stay at high priority ($Q0/Q1$).

Figure 2: I/O-bound process maintaining high priority

Tests equitable CPU time distribution among competing processes.

Figure 3: Fair CPU time distribution among processes

6 Summary

Completed: Implemented MLFQ scheduler with priority-based selection, per-tick time slice tracking, automatic demotion policy, and priority boosting for starvation prevention.

Key Insight: I/O-bound processes maintain high priority by voluntarily yielding before their quantum expires, while CPU-bound processes get demoted for using full time slices.

Next (Week 3): Fine-tune boost interval, add `boostproc()` syscall, comprehensive stress testing.